OPENAI API & CODEX / CONTACT 01

Build an answer you can act on.

RAG. Tool calling. Evidence you can trust.

CUSTOMER MESSAGE

“My parcel arrived late. Can I get compensation?”

Concepts explained · Kora Dispatch as our example

What we'll understand in Contact 1.

01 / GROUND

Embeddings + Qdrant (vector database) → evidence → OpenAI Responses.

02 / CONTINUE + STREAM

Tool calls, conversation context, streamed events and cancellation.

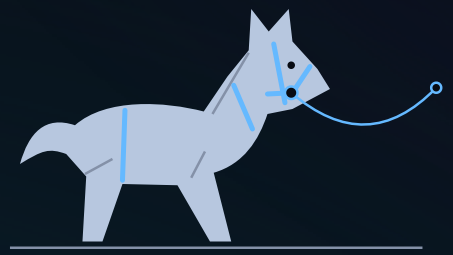
03 / MEASURE

Langfuse traces, golden cases, guardrails, latency and usage.

> One support question. **Understand each part of the AI system.**

The model is the horse.

A harness is the software that runs the model's loop, tools, context and checks.



Model

Capability: interpret, reason, generate.

HARNESS / COORDINATE THE RUN

Loop + tools

Choose a step. Execute permitted tools.

Context + memory

Supply evidence and relevant history.

Checks + stopping

Validate results. Enforce run limits.

> Agent harness = runtime. **Evaluation harness = the test runner.**

Code `workbench/teaching/investigation_reference.py:36` `workbench/teaching/agent_harness.py:84`

Governance sets the rules.

Policies, decision rights and accountability for how AI is used.

THE RULE / GOVERNANCE

Every credit needs supervisor approval.

Who may decide?

ENFORCEMENT / SYSTEM CONTROLS

Block credits without a valid approval.

Enforced by our backend.

Accountable owner — A named supervisor. A reviewable audit record.

> Think rules and enforcement. **People remain accountable.**

Code `workbench/src/relay/service.py:93`

One question. A chain of responsibilities.

Question
Browser

→

Authorize
FastAPI

→

Retrieve + read
RAG & tools

→

Draft
Responses API

→

Validate
Application

> The model advises. **The application authorizes.**

Code `workbench/src/relay/main.py:227`

OpenAI Responses API: the model boundary.

Instructions + input → output items + usage.

create()	Text and tool-call output items.
parse()	SDK parsing into a Pydantic schema.
Validate	Check completion, refusal and citation scope.

```
context = [
  {"role": "developer", "content": RULES},
  {"role": "user", "content": question_and_evidence},
]
answer = client.responses.parse(
  model=MODEL, input=context,
  text_format=GroundedAnswer, store=False,
)
# Check status + output_parsed.
```

> A response can contain tool calls, not just text. **Schema validity does not prove truth.**

Code `workbench/teaching/investigation_reference.py:88` `workbench/teaching/responses_reference.py:14`

RAG has two paths.

01

Prepare

Before the question

Published **sources** → Versioned **chunks** → OpenAI **embeddings** → Qdrant **index**

02

Retrieve

For each question

Embed the **question** → Filter + **search** → Canonical **evidence** → Grounded **answer**

> Embeddings represent meaning as numbers. **Access checks still apply.**

Code `workbench/teaching/contact2/indexing/embeddings.py:26` `workbench/teaching/contact2/querying/retrieval.py:14`

Relevant doesn’t always mean sufficient.

“Why was the driver late?”

RETRIEVED: SERVICE-CREDIT POLICY

Explains **credit eligibility.**

Doesn’t establish **the cause.**

> When the evidence runs out, **abstain and escalate.**

Code `workbench/teaching/investigation_retrieval.py:9` `workbench/teaching/investigation_answer.py:44`

Function calling: request → validate → execute.

01

Select

Function name + JSON arguments

02

Validate

Schema, allowlist, case scope

03

Execute

Authorized read; return call_id

04

Continue

Append output → next response

> Workflow: fixed read, then answer. **Agent: bounded turns, tools and deadline.**

Code `workbench/teaching/agent_harness.py:38` `workbench/teaching/agent_harness.py:84`

Conversation state: who keeps the context?

CALLER-MANAGED / OUR BACKEND

Replay selected history

Send permitted earlier exchanges in `input`. Default: no history.

RESPONSE CHAINING / OPENAI

previous_response_id

Continue from a prior response. Send current instructions again.

CONVERSATIONS API / OPENAI

conversation=id

Attach turns to a persistent provider conversation.

> **Saving ≠ recalling.** Relay rechecks access and source versions. Choose what context the model receives.

Code `workbench/teaching/memory_reference.py:9`

Keep authority outside the prompt.

UNTRUSTED REQUEST

"Approve ₦50,000 now."

SERVER CHECKS

APPLICATION AUTHORITY

- Source access
- Read-only tools
- Credit limits
- Supervisor approval

Code workbench/teaching/agent_harness.py:38 workbench/src/relay/service.py:50

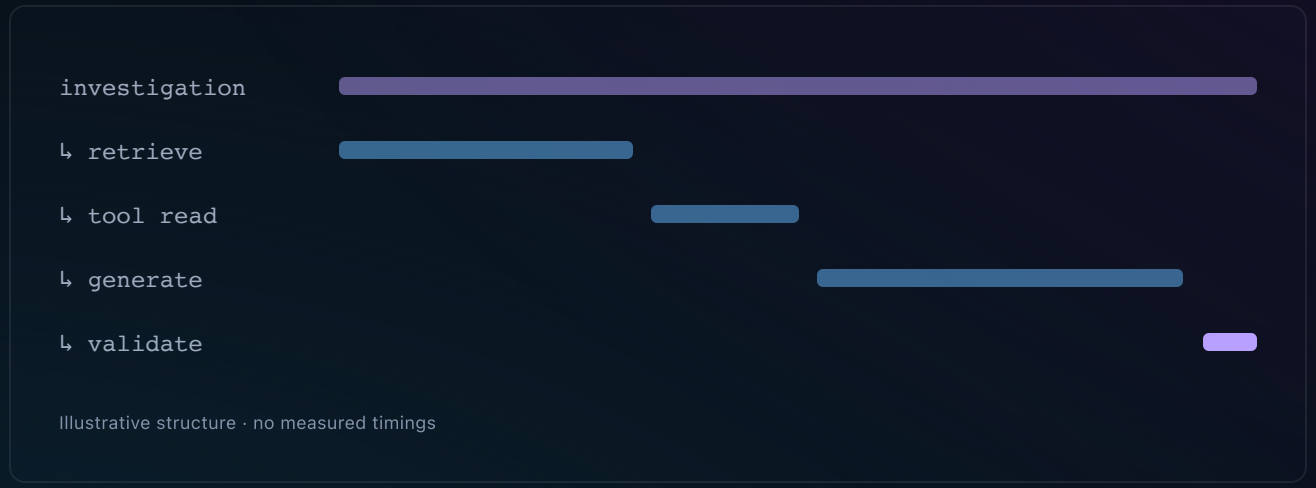
Streaming: events before the final answer.

`response.created`	The provider starts a response.
`response.output_text.delta`	Text arrives incrementally.
`response.completed`	Inspect the final response and usage.

> **Relay streams progress, then a validated answer.** Failure or cancellation must not save partial claims.

Code `workbench/teaching/streaming_reference.py:21` `workbench/teaching/investigation_reference.py:88`

Langfuse: follow one investigation.



Run → spans → evidence.

Prompt/model/corpus versions

Embedding + generation tokens

Latency, failures and request IDs

METADATA EXPORT / NO RAW PAYLOADS

> Local trace ≠ hosted receipt. **Match the run ID in the collector.**

Code `workbench/teaching/observability_reference.py:58`

A good answer needs three kinds of evidence.

01 / RETRIEVAL

Did we find the right source?

02 / ANSWER

Does it support the claim?

03 / EXECUTION

Did we respect the boundary?

> Golden dataset → held-out cases → failure slices → **human review.**

Code `workbench/scripts/evaluate_live.py:30` `workbench/teaching/semantic_evaluation.py:20`

Choose the model and runtime from evidence.

MODEL + CONTEXT

Same task. Measured trade-offs.

Versioned instructions + permitted context.

Output budget: 1,800 tokens per request.

Reasoning / verbosity: model-supported settings.

Inspect cached tokens, p95 latency and cost.

RUNTIME RESPONSIBILITY

FastAPI	Contracts + authority
LangChain	Optional chunking component
LangGraph	Stages + conditional loops
Temporal	Durable workflows / Contact 3

> Freeze corpus, questions and grading. **Configuration options are not measured improvements.**

Code `workbench/teaching/runtime_graph.py:12` `workbench/teaching/provider_settings.py:6`